

Searching Binary Embeddings

Bitplanes and Prefix Backward Walk

13 September 2026

Contents

1	Introduction	2
1.1	Exa method and end to end pipeline	2
1.2	Idea background	2
1.3	Exploring new methods	3
2	Preliminary calculations	11
2.1	Query-time work	12
2.2	Memory	26
2.3	Recall	29
2.4	Summary	33
3	Experiments	34
3.1	Setup	35
3.2	How recall is calculated	35
3.3	Parameter search	36
3.4	Which method is fastest?	37
3.5	Parameters and work	37
3.6	Where does the result change?	38
3.7	Checking the calculations	39
3.8	Code and reproduction	41
4	Limitations and next steps	41
5	Conclusion	42
	References	42

Abstract

I study three ways to search binary document vectors after selecting clusters: a full scan, Bitplanes, and Backward Walk. I compare their work, memory use and retrieval time using the same documents and queries. The experiments examine when the extra index work saves enough full document scores to reduce latency at a required recall.

1 Introduction

1.1 Exa method and end to end pipeline

Let's first understand the "original" pipeline that I am trying to improve upon.

Note that the architecture I am referring to here is based on the pipeline described in Exa's article [1]. I am aware that Exa's actual production system may have changed or improved since then, but for the purpose of this report, let's treat the architecture described in that article as the original baseline that we compare against.

Detail: In this pipeline, document embeddings are first shortened using Matryoshka representations, converted into binary signs, and assigned to clusters. At query time, the query is encoded, a subset of nearby clusters is selected, and the documents inside those clusters are scored using the floating-point query. The top candidates can then be reranked using the original uncompressed vectors.

The main part that I am trying to improve in this report is the search stage after cluster selection. The two new approaches introduced later, Bitplanes and Backward Walk, replace this step while keeping the rest of the pipeline largely the same.

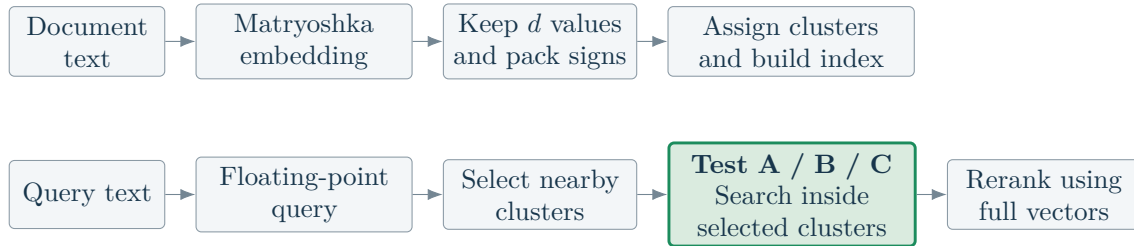


Figure 1: Green marks the part we aim to improve: document search inside the selected clusters, using A, B or C. Reranking is shown as a separate step.

1.2 Idea background

When I took a closer look at the pipeline, one part that particularly stood out to me was the comparison between the floating-point query and the binary-quantized document.

Consider a simple example:

$$q = [0.7, 0.5, -0.4, -0.6, 0.3]$$

and suppose a document is represented using binary signs, where 1 means +1 and 0 means -1. For example:

$$\text{Document} = 11000$$

The score is calculated by taking the dot product between the query values and the document signs. So for 11000, the score is

$$0.7(+1) + 0.5(+1) + (-0.4)(-1) + (-0.6)(-1) + 0.3(-1) = 1.9.$$

Now, something interesting follows from this. Just by looking at the signs of the query, we already know what the ideal binary document would look like.

For every positive query value, we would prefer the corresponding document bit to be 1, and for every negative query value, we would prefer it to be 0. Therefore, for the query above, the ideal binary pattern is

$$11001.$$

This document would achieve the maximum possible score:

$$0.7 + 0.5 + 0.4 + 0.6 + 0.3 = 2.5.$$

From here, we can think about every other document in terms of how far it is from this ideal pattern. More importantly, not every mismatch has the same cost. If we mismatch the first coordinate, where $|q_1| = 0.7$, the score decreases by 1.4. But if we mismatch the last coordinate, where $|q_5| = 0.3$, the score only decreases by 0.6.

In more formal terms, let

$$Q = \sum_i |q_i|$$

be the score of the ideal binary pattern. If a document b disagrees with the query sign at some coordinates, define its mismatch penalty as

$$p = \sum_{i: b_i \neq \text{sign}(q_i)} |q_i|.$$

Then its score can be written as

$$S(q, b) = Q - 2p. \tag{1}$$

This observation is the main idea behind the approaches explored in this report. Instead of treating every binary document equally and scoring all of them one by one, can we use the query itself to guide the search toward documents that are closer to its ideal binary pattern, while avoiding unnecessary full-score calculations?

1.3 Exploring new methods

From the previous section, we know two useful things before we even start searching:

We know what the ideal binary document looks like from the signs of the query. We know that some bits matter more than others, based on the magnitude $|q_i|$.

The next question is therefore:

Can we build an index that uses this information directly, instead of treating every document inside the selected clusters equally?

This led me to explore two approaches.

The first, Bitplanes, starts from a large set of candidate documents and progressively narrows it down using the query.

The second, Backward Walk, approaches the problem from the opposite direction: it starts from a very specific binary pattern and progressively relaxes the constraints until enough useful candidates are found.

1.3.1 Bitplanes

Now that we know what the ideal document looks like based on the query, one idea is to build a data structure that makes filtering by individual bits very cheap.

Normally, binary documents are stored row by row:

ID 2: 10100

ID 3: 11001

ID 4: 11000

Instead, we can additionally store an inverted representation of the same bits:

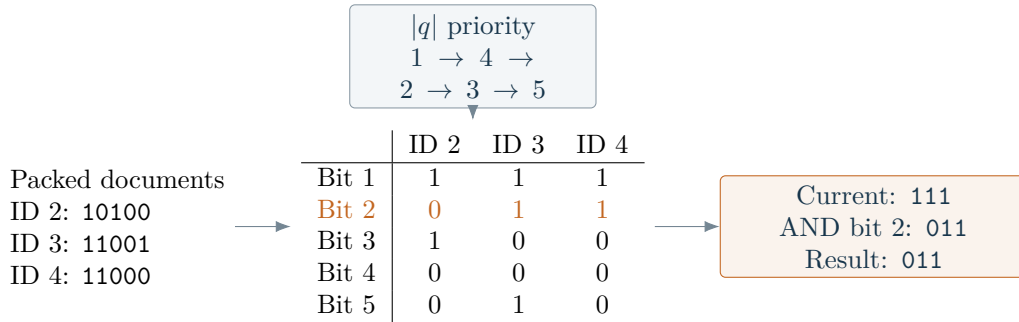


Figure 2: The same document signs, stored in an extra layout for bitwise filtering.

Each row above is what I call a bitplane: one bit position across many documents.

Suppose the query is

$$q = [0.7, 0.5, -0.4, -0.6, 0.3].$$

From the magnitude of the query values, we can prioritize the bits approximately as

$$1, 4, 2, 3, 5,$$

because

$$0.7 > 0.6 > 0.5 > 0.4 > 0.3.$$

Now suppose our current candidate mask is

$$111$$

meaning that IDs 2, 3 and 4 are all still possible candidates.

For bit 2, the query prefers 1. The corresponding bitplane is

$$011$$

so keeping the documents that match the query is simply

$$111 \& 011 = 011.$$

We immediately reduce the candidate set to IDs 3 and 4.

This is attractive because the filtering itself consists mostly of bitwise operations, which modern CPUs can perform very efficiently.

A simplified first version of the idea might therefore look like:

```
def full_score(query, code):
    return sum(q if bit else -q for q, bit in zip(query, code))

def greedy_search(query, cluster, k, leaf_size):
    order = sorted(range(len(query)), key=lambda i: -abs(query[i]))
    active = cluster.full_mask
    for bit in order:
        plane = cluster.planes[bit]
        preferred = plane if query[bit] >= 0 else ~plane
        active &= preferred
        if active.bit_count() <= leaf_size:
            break

    best = TopK(k)
    for row_id, code in cluster.rows_in_mask(active):
        best.add(row_id, full_score(query, code))
    return best.results()
```

However, this simple version has a major problem.

PROBLEM 1: Greedily following the preferred bits can miss the best document

Consider:

$$q = [0.6, 0.5, 0.4, 0.3].$$

The ideal binary document is 1111.

Suppose the database only contains:

Document A: 0111

Document B: 1000

If we greedily start with the first bit, the query prefers 1. We would therefore keep Document B and throw away Document A.

But their complete scores are:

$$S(A) = -0.6 + 0.5 + 0.4 + 0.3 = 0.6$$

while

$$S(B) = 0.6 - 0.5 - 0.4 - 0.3 = -0.6.$$

So Document A is actually better.

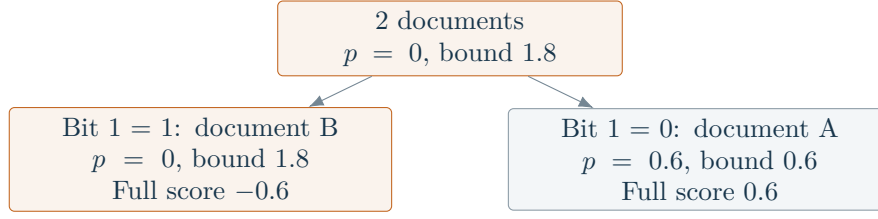
The problem is that matching one important bit does not guarantee that the remaining bits will also match.

Solution: branching

Instead of discarding the alternative, we keep both branches.

The branch matching the preferred bit receives no additional mismatch penalty. The alternative branch is still kept, but its penalty increases by $|q_i|$.

We then explore the branches in order of smallest accumulated penalty. This gives us a recursive search tree rather than one greedy path.



After scoring B, the other bound is $0.6 > -0.6$.
We must explore A; the first choice was not enough.

Figure 3: Keep the alternative until its score bound proves it cannot improve the result.

Stopping condition

Recall:

$$S(q, b) = Q - 2p, \quad Q = \sum_i |q_i|.$$

If a branch has already accumulated mismatch penalty p , any document below it must have penalty at least p .

Therefore,

$$U_{\text{branch}} = Q - 2p$$

is an upper bound on the score of every document inside that branch.

For example, suppose $Q = 2.5$ and we are looking for the top two results. We have already found 2.5 and 1.9, where 1.9 is the current top-2 threshold.

Now consider an unexplored branch with $p = 0.5$. Its absolute best possible score is

$$2.5 - 2(0.5) = 1.5.$$

Since $1.5 < 1.9$, nothing inside that branch can enter our current top two. We can safely discard the entire subtree.

So:

$$Q - 2p < \text{current } K\text{th score} \Rightarrow \text{prune branch}$$

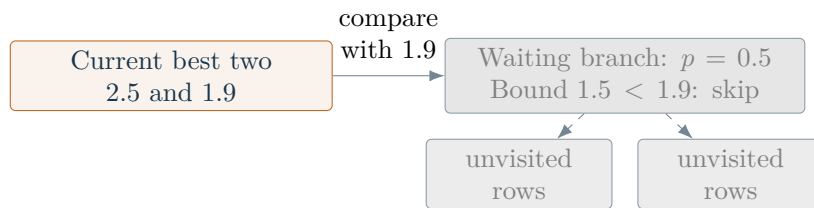


Figure 4: The whole gray subtree can be skipped because even its best possible score is too low.

If the best remaining branch in the queue already fails this test, then every other branch is worse and the entire search can stop.

The resulting algorithm is approximately:

```

from heapq import heappop, heappush
from itertools import count

def bitplane_search(query, clusters, k, leaf_size):
    order = sorted(range(len(query)), key=lambda i: -abs(query[i]))
    ideal_score = sum(abs(q) for q in query)
    sequence = count()
    queue = []
    for cluster in clusters:
        if cluster.full_mask:
            heappush(queue, (0, next(sequence), 0,
                             cluster.full_mask, cluster))

    best = TopK(k)
    while queue:
        penalty, _, depth, active, cluster = heappop(queue)
        upper_bound = ideal_score - 2 * penalty
        if best.full() and upper_bound < best.worst_score():
            break
        if active.bit_count() <= leaf_size or depth == len(query):
            for row_id, code in cluster.rows_in_mask(active):
                best.add(row_id, full_score(query, code))
            continue

        bit = order[depth]
        plane = cluster.planes[bit]
        preferred = plane if query[bit] >= 0 else ~plane
        matching = active & preferred
        # Keep the other half too; its remaining bits may be better.
        children = ((matching, 0), (active ^ matching, abs(query[bit])))
        for mask, extra_penalty in children:
            if mask:
                heappush(queue, (penalty + extra_penalty,
                                 next(sequence), depth + 1, mask, cluster))

    return best.results()

```

This is essentially the branch-and-bound Bitplane search used in the current experiments.

PROBLEM 2: Bitwise operations are cheap, but the bitmap itself can still be large

There is another issue.

A bitwise AND sounds almost free, but it is not literally one operation when the cluster contains thousands or millions of documents.

Suppose one cluster contains 6400 documents. With 64-bit machine words, one bitplane contains

$$6400/64 = 100$$

machine words.

So one split roughly requires us to loop over those 100 words.

Even worse, suppose after several splits only 80 documents remain. The physical mask can still be 100 words wide. So the next split may still need to process those same 100 words.

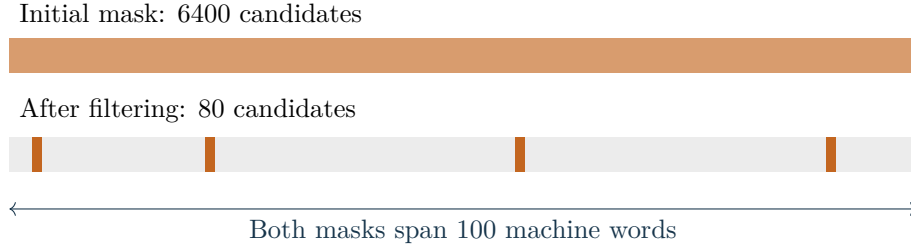


Figure 5: Fewer active document positions do not make this dense mask physically shorter.

Bitmap traversal cost. In general, for a cluster containing n documents, one dense bitplane operation costs roughly $O(\lceil n/64 \rceil)$ word operations. If we visit J split nodes, this becomes approximately $O(J \lceil n/64 \rceil)$, before even counting queue operations and the final document scoring. This was one of the important findings from the earlier experiments: reducing the number of fully scored documents does not automatically mean reducing total query time, because the bitmap traversal itself can become expensive.

Solution 1: Parameter tuning. This problem is technically something we can optimize. For example, we can tune cluster size during coarse routing; number of clusters probed; how deep we split; leaf size; when we switch back to ordinary full scoring. So Bitplanes is not fundamentally broken. The question is finding the point where scores avoided are worth more than additional bitmap traversal.

Solution 2: Other approach. However, thinking about this cost led me to a second idea. Bitplanes starts with a large set of documents and repeatedly narrows it down. What if we instead did the opposite? If Bitplanes can be thought of as bottom-up, then Backward Walk can be thought of as top-down: start from the most specific binary pattern we can describe and progressively broaden the search only when necessary.

1.3.2 Backward Walk

Backward Walk builds on the same observation:

Given a query, we already know what its ideal binary document should look like.

However, instead of starting with thousands of documents and filtering them using bitplanes, we try to start directly from a small bucket of documents that already looks similar to the query.

Suppose the binary query begins with 101101001011....

Using all 256 bits as a lookup key would be far too specific. With a large binary space, the probability that an exact 256-bit pattern actually exists in the database is extremely small.

Instead, suppose we choose a prefix length x . For example, the first 16 bits:

1011010010110101

During index construction, documents sharing this prefix are placed in the same bucket or contiguous range.

At query time, we directly look up this prefix. If it contains enough useful documents, we score them using the complete 256-bit score. If it is empty, or contains too few candidates, we remove one constraint:

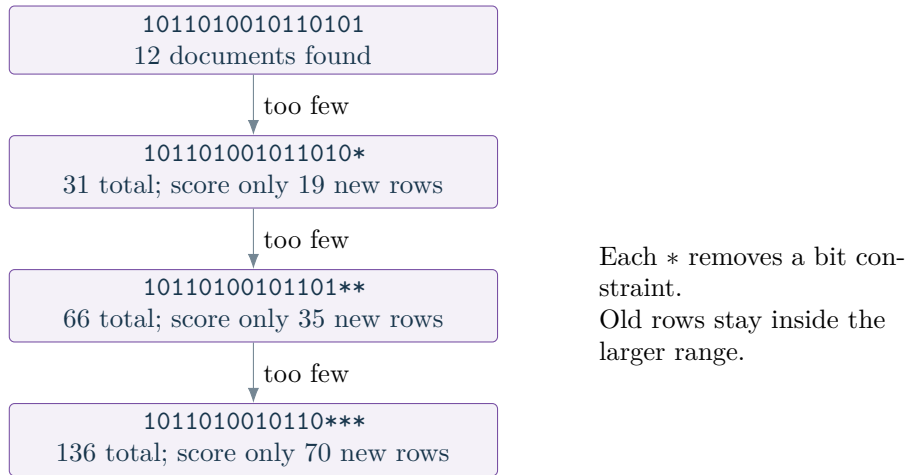


Figure 6: Illustrative counts: $12 + 19 + 35 + 70 = 136$ distinct rows scored, not $12 + 31 + 66 + 136$.

Every step moves to a larger candidate set.

This is why I call the method Backward Walk: instead of progressively adding constraints to a large candidate set, we begin from a very specific location and progressively walk backward toward more general prefixes.

Note that this bit prefix already has an exponential property: if the bits are independent and equally likely to be 0 or 1, each removed bit doubles the expected number of matching documents.

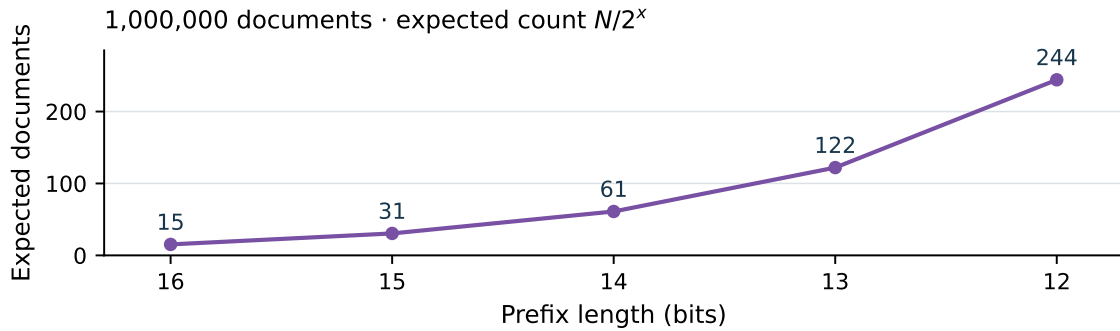


Figure 7: Expected counts for one global cluster; labels are rounded. This estimates rows, not recall.

With a trie or documents ordered by their prefix, the previous range is contained inside the parent range. We only need to process the newly introduced documents.

A simplified query algorithm becomes:

```

def backward_walk(query, clusters, k, start_depth, candidate_target):
    signs = tuple(q >= 0 for q in query)
    previous = [None] * len(clusters)
    best = TopK(k)
    scored = 0
    for depth in range(start_depth, -1, -1):
        for index, cluster in enumerate(clusters):
            left, right = cluster.prefix_range(signs[:depth])
            if previous[index] is None:
                new_ranges = [(left, right)]
            else:
                old_left, old_right = previous[index]
                # Only the new rows beside the old range need scores.
                new_ranges = [(left, old_left), (old_right, right)]
            for start, stop in new_ranges:
                for row_id, code in cluster.rows_in_range(start, stop):
                    best.add(row_id, full_score(query, code))
                scored += stop - start
            previous[index] = (left, right)

        # Finish this depth in every cluster before testing the target.
        if candidate_target > 0 and scored >= candidate_target:
            break
    return best.results()

```

There is one important distinction here.

Stopping simply because we have collected, for example, 100 candidates is an approximate stopping rule. It does not mathematically prove that a better document does not exist outside the current prefix.

Therefore, in the experiments, recall still needs to be measured against the same exact reference results. An exact version would require a valid upper bound on the scores of documents outside the currently visited prefix.

Relationship to HNSW

At first glance, this approach may sound somewhat similar to HNSW because both methods use a hierarchy to progressively change the search space.

However, the hierarchy is being used differently.

HNSW builds a proximity graph. During a query, it begins from sparse upper layers, navigates toward promising neighbors, and progressively descends toward the denser bottom layer [2].

Backward Walk does not need to navigate between neighboring document vectors. The query itself directly tells us where to begin:

query → binary prefix → exact prefix lookup

We then broaden that prefix only when necessary.

So conceptually:

	HNSW	Backward Walk
Index	Proximity graph	Prefix / key hierarchy
Query start	Sparse graph layer	Query's own binary prefix
Movement	Neighbor → neighbor	Specific prefix → broader prefix
Index insertion	Find neighbors + maintain edges	Compute prefix + insert into bucket/range
Search signal	Vector distance	Prefix agreement

Potential advantage 1: cheaper index construction and updates

One possible advantage that I would like to investigate further is the cost of adding new documents. For HNSW, inserting a new vector generally requires searching the existing graph to find suitable neighbors and then creating or updating graph connections.

For Backward Walk, the conceptual insertion process is much simpler:

document $\rightarrow$ embedding $\rightarrow$ binary signs $\rightarrow$ prefix key $\rightarrow$ corresponding bucket

There is no nearest-neighbor graph that needs to be constructed for each new document. So Backward Walk may be attractive for a search index that changes frequently.

I would frame this as a hypothesis to verify, because the actual insertion cost still depends on how the buckets, ranges and on-disk/in-memory index are implemented.

Potential advantage 2: Matryoshka prefixes give the relaxation some meaning

There is also a useful connection with Matryoshka embeddings.

A Matryoshka model is trained so that shorter prefixes of the embedding remain useful representations [3].

Therefore, moving from, for example, 16 bits $\rightarrow$ 15 bits $\rightarrow$ 14 bits is not simply dropping arbitrary dimensions from an ordinary embedding. We are progressively moving to shorter prefixes of a representation that was specifically trained to retain useful information at multiple prefix lengths.

However, this does not mean that the first coordinate is always more important than the second, or that the nearest neighbor under 16 dimensions must remain the nearest neighbor under 15 dimensions.

Matryoshka training gives us a reason to believe that shorter prefixes remain meaningful; the actual recall of each prefix still has to be measured.

2 Preliminary calculations

Before running the experiments, let's first break down where each method actually spends its work.

The idea is simple. Method A scores every document inside the opened clusters. Method B tries to avoid some of those scores by filtering with Bitplanes first. Method C tries to avoid starting from a large set of documents at all by looking up a specific prefix and broadening it only when necessary.

So for each method, I mainly want to understand two things:

What additional work does this method introduce, and how many full document scores can that work potentially save?

I will use the same small example throughout this section and then generalize it into the formulas that we will later compare with the experiments.

Throughout this section:

Symbol	Meaning
N	Total number of documents
C	Total number of clusters
P	Number of clusters opened for one query
d	Number of binary dimensions
K	Number of results we want to return
n_c	Number of documents inside cluster c
F_A, F_B, F_C	Number of documents that receive the full score under A, B and C

If the clusters are roughly balanced, one cluster contains about

$$n \approx \frac{N}{C}$$

documents.

So if we open P clusters, we expose roughly

$$\frac{PN}{C}$$

documents.

This is only an approximation for intuition. The experiments later use the actual number of documents inside the opened clusters.

2.1 Query-time work

Let's use one tiny example so that we can see exactly what each method is doing.

Suppose one opened cluster contains three documents:

ID	Binary document	Full score
0	10100	0.1
1	11001	2.5
2	11000	1.9

The query is

$$q = [0.7, 0.5, -0.4, -0.6, 0.3].$$

From the previous section, we know that the ideal binary document is

11001

and suppose we want the best

$$K = 2$$

documents.

For this example:

$$N = 3, \quad C = P = 1, \quad d = 5.$$

The example is intentionally small. The actual experiments later contain hundreds of thousands or millions of documents.

2.1.1 Method A: scan the opened clusters

Method A is the easiest one to understand.

Once cluster selection has decided which clusters to open, **every document inside those clusters receives the full score.**

Step 1: how many documents do we need to score?

In our example, the only opened cluster contains:

[ID 0, ID 1, ID 2]

so:

$$F_A = 3.$$

More generally, let O be the set of clusters opened for one query. Then:

$$F_A = \sum_{c \in O} n_c$$

is the number of documents A fully scores.

If the clusters are roughly balanced:

$$F_A \approx \frac{PN}{C}.$$

So for A, this is already the most important number: **how many documents did cluster selection leave for us to search?**

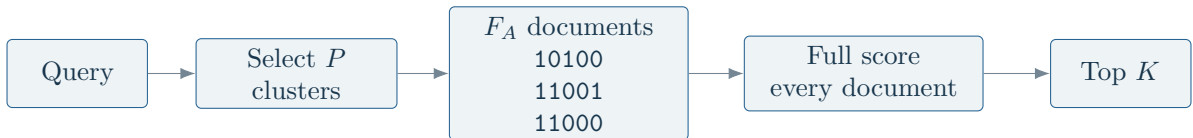


Figure 8: Method A performs no additional filtering after cluster selection. Every document inside the opened clusters receives the full score.

Step 2: what does one full score cost?

The scorer used here follows the four-sign lookup-table approach described in Exa’s article.

Instead of multiplying every query value with every document bit individually, the query is divided into groups of four.

For our five-dimensional example:

$$\begin{bmatrix} 0.7, & 0.5, & -0.4, & -0.6 \\ 0.3, & 0, & 0, & 0 \end{bmatrix}$$

A four-bit group has

$$2^4 = 16$$

possible binary patterns.

So this example needs

$$2 \times 16 = 32$$

precomputed table entries.

In general, the number of groups is

$$G = \left\lceil \frac{d}{4} \right\rceil.$$

One document score therefore reads G table entries.

For our example:

$$G = 2.$$

Three documents therefore require:

$$3 \times 2 = 6$$

table lookups.

In general, the scoring work grows with

$$F_A G.$$

Since G grows with d ,

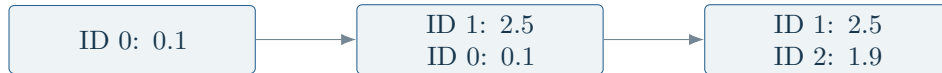
$$T_{\text{score},A} = O(F_A d).$$

For a fixed binary dimension, this is basically linear in the number of documents we score.

Step 3: keep only the best K

As scores are produced, we only need to keep the best K .

For our example:



Using a bounded heap, maintaining the best K results takes at most approximately

$$O(F_A \log(K + 1)).$$

Total cost for A

Let T_{base} contain the work before full document scoring, such as cluster selection and score-table preparation.

Let a_A be the average measured time needed to fully process one document, including its score and the top- K update.

Then:

$$T_A \approx T_{\text{base}} + F_A a_A.$$

For roughly balanced clusters:

$$T_A \approx T_{\text{base}} + \frac{PN}{C} a_A.$$

The main tradeoff is straightforward.

Using more clusters usually makes each cluster smaller, so each opened cluster contains fewer documents. However, we may then need to open more clusters to maintain the same recall.

TL;DR — Method A: A's cost is mainly determined by how many documents cluster selection leaves behind. Fewer opened documents means fewer full scores, but opening too few clusters can hurt recall.

2.1.2 Method B: Bitplanes

Bitplanes try to reduce the number of documents that receive the full score.

Instead of immediately scoring every document inside the opened clusters, B first uses Bitplanes to filter the documents.

So B has two main costs:

$$\text{Bitplane work} + \text{full scoring of the documents that survive}$$

The whole idea only makes sense if the full scores we avoid are more expensive than the Bitplane work required to avoid them.

Step 1: inspect the important query bits first

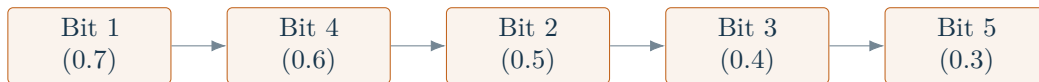
B uses the same cluster selection and the same full-score table as A.

The additional step is to order the query bits by decreasing $|q_i|$.

For

$$q = [0.7, 0.5, -0.4, -0.6, 0.3],$$

the order is:



We inspect the larger values first because mismatching them hurts the score more.

Sorting these positions costs

$$O(d \log d).$$

We also calculate the ideal score:

$$Q = \sum_i |q_i| = 2.5.$$

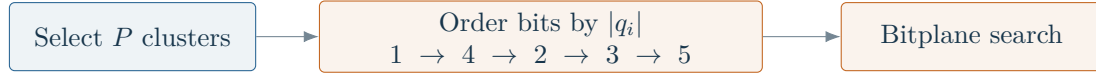


Figure 9: Bitplanes keep the same cluster-selection stage as A, but add query-dependent filtering before the full scorer.

Step 2: represent the documents using a mask

Inside one cluster, B keeps a bitmap showing which document positions are still active.

If cluster c contains n_c documents, its mask needs

$$W_c = \left\lceil \frac{n_c}{64} \right\rceil$$

64-bit machine words.

Our three-document example fits inside one word.

The starting mask is:

111

which means that all three documents are still active.

Step 3: split using the query bits

The first query position is bit 1, and the query prefers 1.

All three documents match:

$$111 \text{ AND } 111 = 111$$

so nothing is removed.

The next position is bit 4. The query prefers 0, and again all three documents match.

So the mask is still:

111.

The next position is bit 2.

Its Bitplane is:

011.

Therefore:

$$111 \text{ AND } 011 = 011$$

The preferred branch now contains IDs 1 and 2.

The alternative branch is:

100

which contains ID 0.

Because ID 0 disagrees with the query at bit 2, this branch has mismatch penalty:

$$p = |q_2| = 0.5.$$

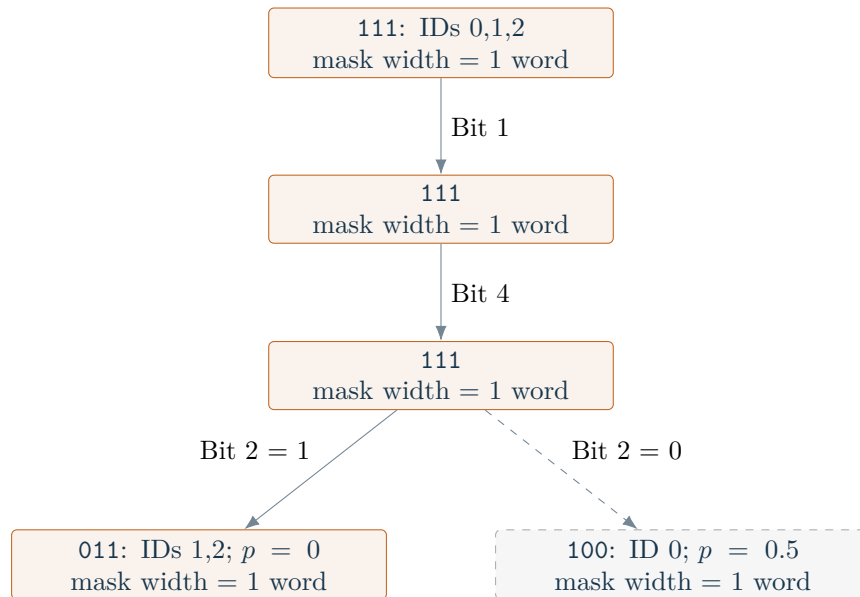


Figure 10: The first two bit tests remove no documents. The third separates ID 0 into another branch, but that branch is not discarded yet.

Step 4: why Bitplane filtering is not free

At first, a bitwise AND sounds almost free.

But that is only true for one machine word.

Suppose one cluster contains:

$$6,400$$

documents.

One mask then needs:

$$6400/64 = 100$$

machine words.

So one split does not perform one AND. It loops across roughly 100 word positions.

There is another important detail: **the mask does not physically get shorter when fewer documents remain active.**

For example, suppose only 80 documents survive after several splits.

The mask can still be 100 words wide.

So another split may still need to scan all 100 words.

This is the main cost that B introduces.

Step 5: count the Bitplane work

Let J_c be the number of times B splits a mask in cluster c .

Each split scans W_c words.

So the total amount of split-mask work is:

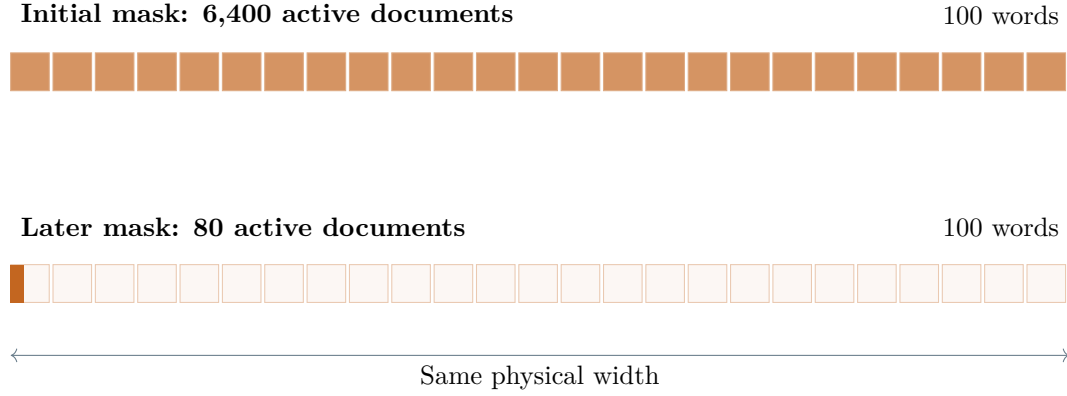


Figure 11: The number of active documents can become much smaller while the physical bitmap stays the same size.

$$V_{\text{split}} = \sum_{c \in O} J_c W_c.$$

For roughly balanced clusters, if B performs an average of S splits per opened cluster:

$$V_{\text{split}} \approx PS \left\lceil \frac{N}{64C} \right\rceil.$$

Step 6: eventually stop splitting and score the documents

B does not split forever.

Once a branch contains few enough documents, it becomes a **leaf**, and B fully scores the documents inside it.

Suppose our leaf size is two.

The preferred branch:

011

contains exactly IDs 1 and 2.

So B calculates:

$$\begin{aligned} \text{ID 1} &\rightarrow 2.5 \\ \text{ID 2} &\rightarrow 1.9 \end{aligned}$$

Therefore:

$$F_B = 2.$$

Method A would have scored all three documents.

Leaf masks still need to be read

Before B can extract the surviving document positions, it still has to scan the leaf mask.

Let H_c be the number of leaf masks read in cluster c .

Then:

$$V_{\text{leaf}} = \sum_{c \in O} H_c W_c.$$

I keep split-mask work and leaf-mask work separate because the implementation measures both separately.

Step 7: can we skip the waiting branch?

ID 0 is still waiting.

Its known mismatch penalty is:

$$p = 0.5.$$

The best possible score inside that branch is:

$$Q - 2p = 2.5 - 2(0.5) = 1.5.$$

Our current top two scores are:

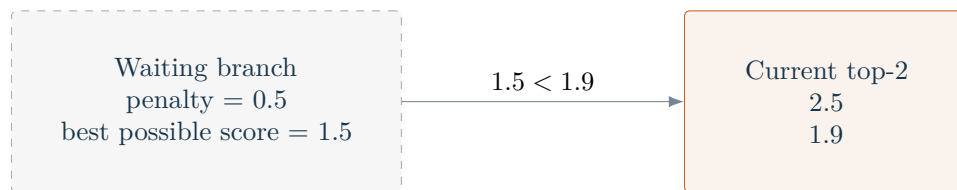
$$2.5, \quad 1.9.$$

Since:

$$1.5 < 1.9,$$

nothing inside the waiting branch can enter the current top two.

So B can safely skip it.



→ skip branch

Figure 12: Once the best possible score of a branch falls below the current top- K threshold, the whole branch can be skipped.

Total cost for B

B therefore pays for:

- ordering the query bits;
- reading split masks;
- reading leaf masks;
- maintaining the waiting branches;
- fully scoring F_B documents.

Let:

Symbol	Meaning
b_s	Average time for one split-mask word
b_l	Average time for one leaf-mask word
T_{order}	Ordering query bits and preparing query weights
T_{queue}	Managing branches, masks and the priority queue
a_B	Average time to fully process one surviving document

Then:

$$T_B \approx T_{\text{base}} + T_{\text{order}} + b_s V_{\text{split}} + b_l V_{\text{leaf}} + F_B a_B + T_{\text{queue}}.$$

The key condition is:

$$\text{full-score work avoided} > \text{Bitplane work added.}$$

A smaller leaf size may reduce F_B , because B keeps filtering longer. However, that also means more splits and more bitmap work.

A larger node budget can also improve recall, but again it allows B to search more branches.

TL;DR — Bitplanes: B is useful when a relatively small amount of bitmap work can eliminate a large number of full document scores. Scoring fewer documents alone is not enough; the bitmap work also has to be cheap enough.

2.1.3 Method C: Backward Walk

Backward Walk starts from the opposite direction.

A begins with every document inside the opened clusters.

B also starts with all of them, then progressively removes documents.

C instead starts from a **very specific query prefix** and progressively broadens it only when necessary.

So C mainly pays for:

$$\text{prefix lookups} + \text{full scoring of the documents that become visible}$$

Step 1: store the documents in prefix order

Using the same example, sort the documents by their binary key:

Position	Binary key	ID
0	10100	0
1	11000	2
2	11001	1

The query's ideal binary key is:

11001.

Step 2: start from the most specific prefix

Suppose C starts using all five bits.
It first searches for:

11001.
Because the documents are sorted, two binary searches find the beginning and end of this prefix range.
The result is:
[2,3)
which contains only ID 1.
So C scores:

$ID\ 1 \rightarrow 2.5$

We need two results, so C continues.

Step 3: broaden the prefix

C removes one prefix constraint:

$$\begin{array}{c} 11001 \\ \downarrow \\ 1100* \end{array}$$

The new range is:
[1,3)
which contains IDs 2 and 1.
ID 1 was already scored.
So C only needs to score the newly added document:
ID 2 $\rightarrow$ 1.9.
Now C has two results.

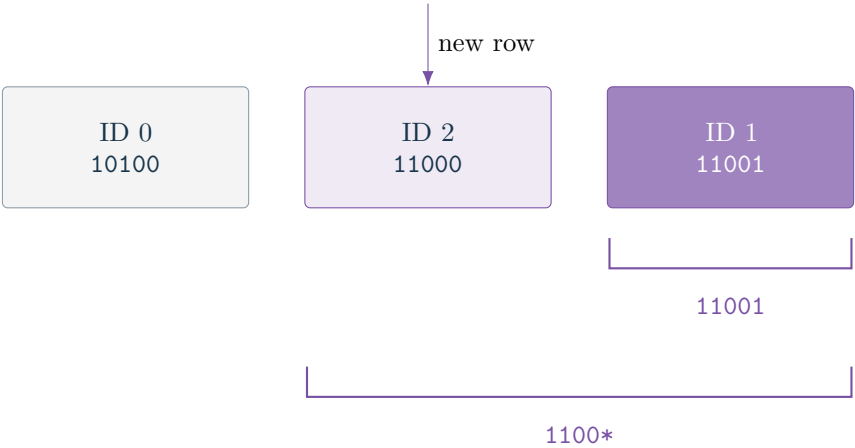


Figure 13: A broader prefix already contains the documents found at the previous depth, so only the newly exposed rows need to be scored.

This is important because C does not rescore the same documents every time the prefix becomes shorter.

Step 4: how many prefix lookups do we perform?

Let:

- x : starting prefix depth;
- ℓ : depth where C finally stops.

Then the number of depths visited is:

$$L = x - \ell + 1.$$

Depth zero represents the entire cluster, so there is no prefix lookup there.

Let L_+ be the number of visited depths above zero.

At each positive depth, C performs two boundary searches in each opened cluster:

- one to find the start of the matching prefix;
- one to find the end.

Therefore the number of binary-search calls is:

$$\boxed{2PL_+}.$$

For cluster c , each search costs approximately:

$$O(\log(n_c + 1)).$$

So the lookup work is approximately:

$$\boxed{2L_+ \sum_{c \in O} \log_2(n_c + 1)}.$$

Step 5: how many documents end up receiving the full score?

Let:

$$F_C$$

be the total number of distinct documents exposed before C stops.

Because every broader prefix contains the previous range, each document is scored at most once.

So the full-score part is simply:

$$F_C a_C.$$

This means the most important question for C is:

How broad does the prefix have to become before we have enough useful documents?

Step 6: how quickly does a prefix grow?

For a simple intuition, temporarily assume that every bit is:

- independent;
- equally likely to be 0 or 1.

Then a prefix of length ℓ matches one random document with probability:

$$2^{-\ell}.$$

So in a cluster with n documents, the expected number of matches is:

$$\frac{n}{2^\ell}.$$

Across P roughly balanced clusters:

$$E[F_\ell] \approx \frac{PN}{C2^\ell}.$$

For one global collection of one million documents:

Prefix depth	Expected matching documents
16	about 15
15	about 31
14	about 61
13	about 122
12	about 244

So removing one prefix bit roughly doubles the number of matching documents under this simplified model.

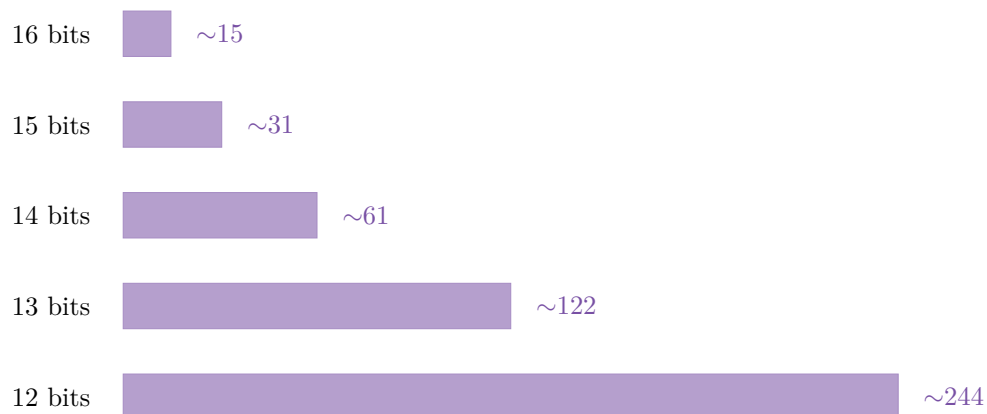


Figure 14: Under a simple balanced-bit model, removing one prefix constraint approximately doubles the number of matching documents.

This is only an intuition for **how many documents** a prefix may contain.

It is not a recall estimate.

Real embedding bits can be correlated, and the distribution can change further once the documents have been divided into clusters.

Step 7: when should C stop?

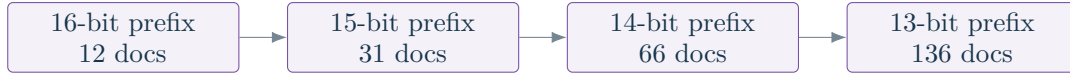
There are two possible stopping rules.

Approximate stopping

The simplest rule is:

stop once at least M documents have been found.

For example:



If our target is 100 documents, we stop at 13 bits.

This is cheap, but it does **not** prove that a better document does not exist outside that prefix.

So this version can lose recall.

Exact stopping inside the opened clusters

We can also use the score equation to prove that no unseen document can beat the current results.

Suppose C has fully searched a prefix of length ℓ .

Any unseen document must disagree with the query on at least one of those first ℓ bits.

The cheapest possible mismatch is:

$$m_\ell = \min_{1 \leq i \leq \ell} |q_i|.$$

Therefore the highest possible score of any unseen document is:

$$U_\ell = Q - 2m_\ell.$$

Let τ be the worst score currently inside the top- K .

If:

$$\tau > U_\ell,$$

then no unseen document inside the opened clusters can enter the result.

C can stop safely.

If that never happens, C can keep relaxing the prefix until depth zero, where it has exposed every document inside the opened clusters.

At that point C becomes equivalent to A locally.

Total cost for C

Let:

Symbol	Meaning
T_{key}	Preparing the query prefix and score-bound data
t_b	Average time for one binary-search comparison
T_{ranges}	Updating prefix ranges and identifying new rows
a_C	Average time to fully process one exposed document

Then:

$$T_C \approx T_{\text{base}} + T_{\text{key}} + 2L + t_b \sum_{c \in O} \log_2(n_c + 1) + F_C a_C + T_{\text{ranges}}.$$

The key condition is:

$$\text{full-score work avoided} > \text{prefix lookup work added.}$$

If C can stop at a deep prefix, then:

$$F_C \ll F_A$$

and it may save a lot of scoring work.

But if C has to relax all the way to depth zero:

$$F_C = F_A,$$

so it does all of A's scoring **plus** the additional prefix lookups.

TL;DR — Backward Walk: C works best when a fairly specific query prefix already contains the documents we need. If it has to keep broadening until almost the whole cluster is visible, then the prefix index does not help.

2.1.4 Comparing the three methods

At this point, the difference between the methods can be summarized very simply.

Method	What it spends time on
A	Score every document inside the opened clusters
B	Bitplane filtering + score the documents that survive
C	Prefix lookups + score the documents that become visible

The approximate time models are:

Method	Approximate query time
A	$T_{\text{base}} + F_A a_A$ $T_{\text{base}} + T_{\text{order}}$
B	$+ b_s V_{\text{split}} + b_l V_{\text{leaf}}$ $+ F_B a_B + T_{\text{queue}}$
C	$T_{\text{base}} + T_{\text{key}} + T_{\text{lookup}}$ $+ F_C a_C + T_{\text{ranges}}$

where:

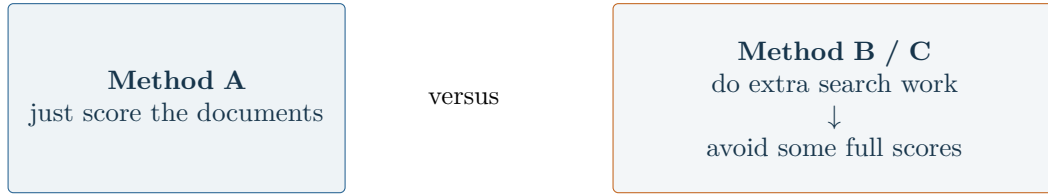
$$V_{\text{split}} = \sum_{c \in O} J_c W_c,$$

$$V_{\text{leaf}} = \sum_{c \in O} H_c W_c,$$

and:

$$T_{\text{lookup}} = 2L_+ t_b \sum_{c \in O} \log_2(n_c + 1).$$

The overall question is therefore:



The new method is only worthwhile if the second path ends up cheaper.

2.2 Memory

Query time is not the only cost.

B and C also require additional index structures, so we need to understand how much memory each method stores.

There are two useful categories:

Persistent index memory is built before queries arrive and remains loaded.

Temporary query memory only exists while one query is running.

2.2.1 Method A

A stores:

- one packed binary code per document;
- one document ID per document;
- the cluster/routing information.

With 64-bit words, one binary code occupies:

$$8 \left\lceil \frac{d}{64} \right\rceil$$

bytes.

An ID occupies 8 bytes.

So the document rows use:

$$M_{A,\text{rows}} = N \left(8 \left\lceil \frac{d}{64} \right\rceil + 8 \right).$$

The router uses additional memory.

For example, C float32 centroids of routing dimension d_R use approximately:

$$4Cd_R$$

bytes, plus their labels, offsets and implementation overhead.

During one query, A also keeps:

- the floating-point query;
- the score table;
- the current top- K ;
- small routing state.

This is the baseline memory that B and C build on.

2.2.2 Method B

B keeps everything A stores.

It additionally stores:

1. the Bitplanes;
2. the temporary branch masks used during search.

Stored Bitplanes

One Bitplane in cluster c uses W_c words.

There are d Bitplanes.

Therefore:

$$M_{\text{planes}} = 8d \sum_{c=1}^C W_c.$$

The ceiling in W_c matters because every cluster has its own word padding.

Temporary branch masks

Every active or waiting branch has its own mask.

For cluster c , one mask uses:

$$8W_c$$

bytes.

If several branches are waiting, several full-width masks can be alive at the same time.

If $A_c(t)$ masks from cluster c are alive at time t , then the raw peak mask memory is:

$$Q_{\text{masks}} = 8 \max_t \sum_c A_c(t) W_c.$$

The priority queue and node information require additional memory.

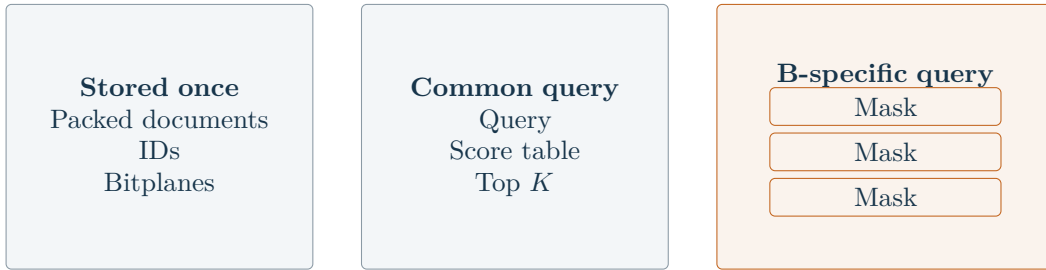


Figure 15: B stores an additional Bitplane layout and can also keep several full-width masks alive during one query.

TL;DR — B memory: B uses more persistent memory because it stores the bits twice: once by document and once as Bitplanes. Its main temporary-memory cost comes from the masks of waiting branches.

2.2.3 Method C

C also keeps A's binary documents and IDs.

The current implementation additionally stores one 32-bit prefix key per document.

So:

$$M_{\text{prefix}} = 4N.$$

The documents are sorted once by this key inside each cluster.

Importantly, C does **not** store a new copy of the documents for every prefix length.

The same sorted rows support:

```
11001
1100*
110**
11***
...
```

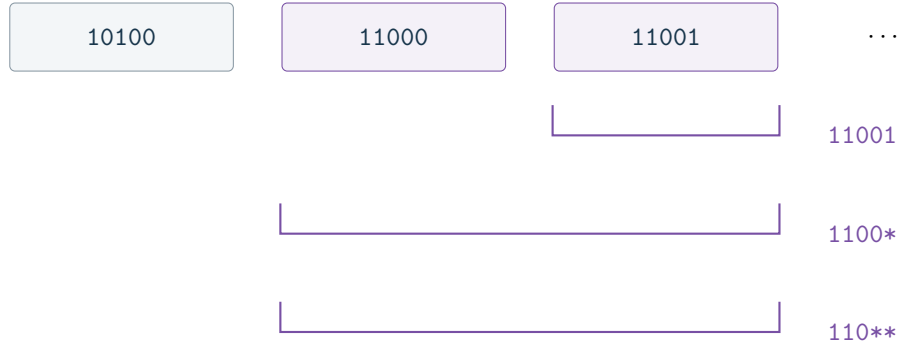


Figure 16: Different prefix lengths point to different ranges of the same sorted document array. The documents are not copied for every prefix.

by using different ranges.

During one query, C mainly keeps the previous range for every opened cluster.

So its temporary search state is relatively small compared with B.

TL;DR — C memory: C adds one small prefix key per document, but it does not need full-document bitmaps or many branch masks during the query.

2.2.4 Memory comparison

Method	Persistent index	Main temporary query memory
A	Binary documents + IDs + router	Query, score table, top- K
B	A + Bitplanes	Waiting branch masks
C	A + prefix keys	Previous prefix ranges

The formulas above count the main logical fields.

The actual process memory can be somewhat larger because of allocation capacity, runtime state and temporary buffers.

For that reason, the experiments later report both the calculated index size and the measured process peak.

2.3 Recall

So far we have only talked about speed.

But avoiding work is only useful if we still return the correct documents.

For query q , let:

- G_q be the exact global top- K document IDs;
- $O_{a,q}$ be the IDs returned by method a .

Then average recall is:

$$R_a = \frac{\sum_q |G_q \cap O_{a,q}|}{n_q K}.$$

For example:

Reference:
[1, 2]
Returned:
[1, 0]

recovers one out of two:

$$Recall = 50\%.$$

Where can recall be lost?

There are actually two stages where a correct document can disappear.

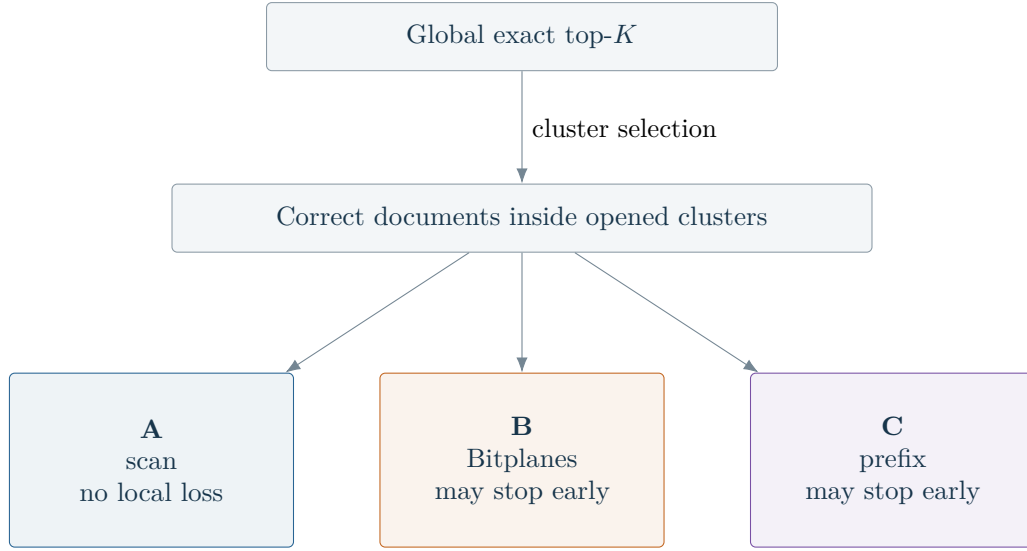


Figure 17: Every method can miss a global result if its cluster was not opened. B and C can introduce another loss if their local search stops before reaching that document.

2.3.1 Method A

Let:

$$g_q$$

be the number of global reference documents whose clusters were opened.

A scores every document in those clusters.

So all g_q of those reference documents are recovered locally.

Therefore:

$$R_{A,q} = \frac{g_q}{K}.$$

A can lose recall during cluster selection, but not during its local scan.

2.3.2 Method B

Suppose B recovers fraction:

$$r_{B,q}$$

of the reference documents that are already inside the opened clusters.

Then:

$$R_{B,q} = \frac{g_q}{K} r_{B,q}.$$

For example, suppose:

- 8 of the 10 correct documents are inside the opened clusters;
- B finds 6 of those 8.

Then:

$$Recall = \frac{8}{10} \times \frac{6}{8} = 60\%.$$

If B performs a complete branch-and-bound search using valid score bounds, then:

$$r_{B,q} = 1.$$

If we give B a finite node budget, it may stop early and reduce recall.

TL;DR — B recall: Routing can lose documents before B even starts, and a limited Bitplane search can lose additional documents inside the opened clusters.

2.3.3 Method C

Let:

$$D_{q,\ell}$$

be all documents exposed when Backward Walk stops at prefix depth ℓ .

Every document in that set has received the full score.

So:

$$R_{C,q} = \frac{|G_q \cap D_{q,\ell}|}{K}.$$

The important point is:

The number of documents inside a prefix does not tell us its recall.

A prefix can contain many documents and still miss the best result.

Small example

Take:

$$q = [0.1, 0.9].$$

The ideal binary pattern is:

11.

Suppose the only two documents are:

10

and:

01.

Their scores are:

$$S(10) = 0.1 - 0.9 = -0.8,$$

$$S(01) = -0.1 + 0.9 = 0.8.$$

If C searches:

1*

it finds 10.

But the better document is 01, which lies outside that prefix.

So simply finding one document does not prove that we found the best document.

When is C exact inside the opened clusters?

C can become exact in two ways.

The simple way is to keep relaxing until depth zero.

At depth zero, every document inside the opened clusters has been exposed, so C becomes equivalent to A locally.

The second way is to prove that the unseen documents cannot win.

At prefix depth ℓ , every unseen document must disagree with the query in at least one of the first ℓ bits.

The cheapest such mismatch is:

$$m_\ell = \min_{1 \leq i \leq \ell} |q_i|.$$

So the best possible score of any unseen document is:

$$U_\ell = Q - 2m_\ell.$$

Let τ be the worst score currently inside the top- K .

If:

$$\tau > U_\ell,$$

then no unseen document inside the opened clusters can enter the result.

C can stop safely.

TL;DR — C recall: Stopping after “enough documents” is only an approximation. C is exact locally only when it has either exposed every document or mathematically proved that the unseen documents cannot beat the current top- K .

2.4 Summary

Query time

Method	Approximate query cost
A	$T_{\text{base}} + F_A a_A$ $T_{\text{base}} + T_{\text{order}}$
B	$+ b_s V_{\text{split}} + b_l V_{\text{leaf}}$ $+ F_B a_B + T_{\text{queue}}$
C	$T_{\text{base}} + T_{\text{key}} + T_{\text{lookup}}$ $+ F_C a_C + T_{\text{ranges}}$

with:

$$F_A = \sum_{c \in O} n_c,$$

$$V_{\text{split}} = \sum_{c \in O} J_c W_c,$$

$$V_{\text{leaf}} = \sum_{c \in O} H_c W_c,$$

$$T_{\text{lookup}} = 2L_+ t_b \sum_{c \in O} \log_2(n_c + 1).$$

Memory

Method	Stored index	Main temporary query memory
A	M_0	Q_0
B	$M_0 + M_{\text{planes}}$	$Q_0 + Q_B$
C	$M_0 + M_{\text{prefix}}$	$Q_0 + Q_C$

Recall

Method	Per-query recall	When local search is exact
A	g_q/K	Always, because every opened document is scored
B	$(g_q/K)r_{B,q}$	Complete safe Bitplane search
C	$ G_q \cap D_{q,\ell} /K$	Reach depth zero or prove unseen documents cannot win

Main variables

Shared

Symbol	Meaning
N	Total number of documents
C	Total number of clusters
P	Number of clusters opened for one query
n_c	Documents inside cluster c
d	Binary dimensions
K	Number of requested results
O	Set of opened clusters
F_A	Documents fully scored by A

Bitplanes

Symbol	Meaning
W_c	Machine words in one mask for cluster c
J_c	Number of splits performed in cluster c
H_c	Number of leaf masks read in cluster c
V_{split}	Total split-mask word reads
V_{leaf}	Total leaf-mask word reads
F_B	Documents fully scored by B

Backward Walk

Symbol	Meaning
x	Starting prefix depth
ℓ	Final prefix depth
L_+	Number of positive prefix depths searched
F_C	Distinct documents fully scored by C
$D_{q,\ell}$	Documents exposed at the final prefix depth

The main idea from the entire section is therefore simple:

A just pays for full scores. B and C spend some extra work in an attempt to avoid those scores. The experiment will tell us whether the scores they avoid are actually expensive enough to make that extra work worthwhile.

3 Experiments

The previous section describes the work each method performs. Here I vary the index and search settings, compare measured query time at the same recall requirement, and inspect the work counts to understand the results.

Each method can choose its own parameters. Within a comparison, the document codes, queries, score, reference IDs, recall requirement and RAM allowance stay fixed. The question is: at that same recall requirement, what is the fastest configuration we tested for each method?

3.1 Setup

I use two collections and several supported code lengths from their cached embeddings. Every method sees all 1,000 fixed queries in a pool. A different model or code length is a separate comparison:

Collection	Embedding model	Documents	Queries	Bits
MS MARCO	Nomic v1.5	1,000,000	1,000	64, 256, 768
Quora (BEIR)	Qwen3 0.6B	522,931	1,000	32, 256, 1024

I reused the cached Nomic document embeddings and selected 1,000 queries from MS MARCO’s official `dev.small` file [6, 4]. For Quora, I encoded the full BEIR corpus and selected 1,000 official test queries [7, 5]. Both random selections were fixed before search. Corpus duplicates remain; none of the selected Quora queries has identical text or ID in its corpus.

Each model produces a full vector once. I take supported prefixes, normalize them, and store their signs. Each length has its own reference: score every document and keep the best 100, resolving ties by document ID. The first one or ten IDs give the smaller result sets. Recall here measures recovery of these binary-score references; human relevance labels are a separate measure.

The machine is an Apple M5 Max with 128 GiB of memory. I use one native search query at a time and one Faiss search thread. The completed runs used a common 32 GB process cap. This is a recorded experiment setting, not the budget required by the formulas; we can evaluate other budgets such as 64 or 128 GB. The largest peak among the selected repeated runs was 505.6 MB, so none of those selections was close to the 32 GB cap. Reported query time starts with a cached query vector and ends with the returned IDs. It includes selecting clusters and searching their documents. Embedding generation and text reranking are outside this timer. Encoding and reference preparation finish before timed searches begin.

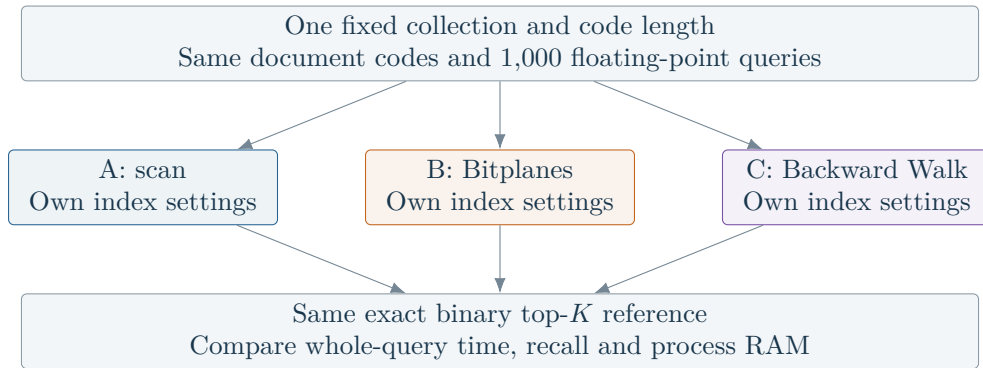


Figure 18: Only index and search settings differ within a pool. Changing the code length creates a new pool with its own reference ranking.

3.2 How recall is calculated

The recall in these experiments is measured from returned document IDs. It is not estimated by assuming a normal distribution or independent bits.

For each query, I first score every document in the collection using the floating-point query and binary document codes. I keep the best K IDs as the reference, resolving equal scores by document ID. A, B and C are then compared with that same reference.

For example, suppose we ask for five results:

Exact scan’s top 5	[12, 7, 91, 4, 30]
Method returns	[12, 7, 91, 4, 85]
Reference IDs recovered	[12, 7, 91, 4]

We recovered four of the five reference IDs, so

$$\text{Recall@}K = \frac{\text{reference IDs recovered}}{K}, \quad \text{Recall@5} = \frac{4}{5} = 80\%.$$

The table values average this calculation over all 1,000 queries. For $K = 1$, recovering the exact best ID on 990 of those queries gives 99% recall. Result order does not affect this overlap count.

What does that percentage mean? A value of 99% means recovering 99% of the exact binary top- K IDs on average. It does not mean finding 99% of all human-relevant documents, and it does not establish agreement with the original full-float embedding. Those are separate evaluations. Each code length has its own binary reference.

Does it measure only search inside a cluster? The current reference covers the whole collection. A reference document counts as missed whether its cluster was never opened or the local search skipped it after opening that cluster. The reported recall therefore includes both sources of loss.

To isolate search inside selected clusters, we would instead obtain the reference by scanning those same selected clusters completely. B and C could then be compared against that local reference. The existing tables use the whole-collection reference, not this isolated measurement.

3.3 Parameter search

For A, I vary the cluster layout, cluster count C and probes P . The reference-cluster ranks give probe choices at the required recall targets. B receives those choices along with leaf sizes and node budgets. C receives them along with starting depths, candidate targets and the optional exact-stop rule.

Settings	Tested ranges and follow-ups
Routing	Float-trained clusters; binary-trained layouts also available to all methods in the short-code follow-ups. Cluster counts include 1, 16, 64, 256, 1024 and 4096.
Result count and recall	$K = 1, 10, 100$; targets 50%, 80%, 90%, 95%, 99%, with additional probes near high recall and whole-collection controls.
B	Initial leaves 128/2048 and budgets 512/8192; shorter codes also use leaves 8/32 and deeper-first ties. Probability 0/0.1/0.2 is a separate three-seed comparison.
C	Starting depths 0/1/4/16/32; candidate targets 1,000/10,000/100,000/half the collection/0. Exact-stop runs use target zero.

This is a summary of the settings tried, not a full cross-product at every width. The two 256-bit grids each have 570 settings; wider Qwen keeps $K = 1, 100$, and full-width Nomic focuses on $K = 100$ at 95% and 99% recall. Across the runs, 3,679 of 4,024 A/B/C settings qualify, plus all 72 probability settings. Three grids remain partial under their recorded time limits. The saved schedules list every attempted and missing setting.

I record complete-query time, recall, process RAM and the method’s work counts. For each target, I select the fastest qualifying setting in the main run, then repeat that exact choice in three fresh processes with varied run order. The tables report those repeated observations. Failed recall targets and incomplete runs stay in the saved data.

I also check direct alternatives. If B visits 512 clusters and scores all their rows without splitting, A must be allowed to use 512 probes too. C gets a depth-zero scan at A’s selected routing settings. These controls prevent a gap in the parameter grid from looking like an algorithmic advantage.

The equations identify useful quantities to inspect, such as the scored fraction and mask width. They do not turn the broad grid into a proven optimum, or supply the measured time constants in advance.

3.4 Which method is fastest?

The first table shows Qwen32 on all 522,931 Quora documents, returning one result. Each cell gives the repeated median query time and achieved recall. Bold marks the lowest median among methods that meet the row’s requirement.

Required recall	A (ms / recall)	B (ms / recall)	C (ms / recall)	Lowest median
80%	0.0308 / 80.3%	0.0295 / 81.9%	0.0318 / 80.3%	B
90%	0.0498 / 90.1%	0.0432 / 90.6%	0.0497 / 90.1%	B
95%	0.0805 / 95.0%	0.0520 / 95.3%	0.0799 / 95.0%	B
99%	0.1948 / 99.0%	0.0809 / 99.2%	0.1688 / 99.0%	B

B has the lowest repeated median at these four targets. The strongest difference is at 99%: 0.0809 ms for B versus 0.1948 ms for A, about 2.4 times faster. These recalls measure the 32-bit ranking. Even its exact top-1 answer agrees with the full-float best score on only 14% of queries; the code-length comparison below measures that separate quality tradeoff.

522,931 documents · 1,000 queries · 32 bits

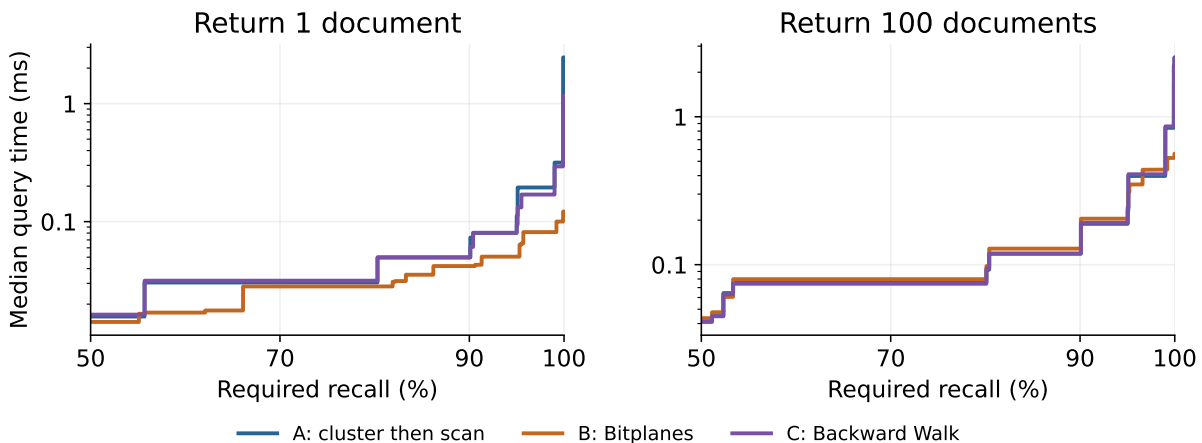


Figure 19: Qwen32 recall and query time, starting from cached vectors. The vertical axes use a log scale. The curves choose the fastest completed settings meeting each recall; the tables use their repeated timings.

3.5 Parameters and work

At the 99% target, each method chooses a different routing layout. The selected configurations are:

Method	Clusters	Probes	Local setting	Recall	ms
A	4096	180	Scan opened rows	99.00%	0.1948
B	16	9	Leaf 128; budget 8192; deeper ties	99.20%	0.0809
C	1024	78	Start 4; target 0; exact stop	99.00%	0.1688

Time is the median over the three repeated processes’ 3,000 query observations. B’s process medians range from 0.08025 to 0.08167 ms, below A’s 0.19223–0.19708 ms and C’s 0.16781–0.16994

ms. B’s finite budget gives measured 99.2% recall, not an exactness guarantee. C proves the answer within its opened clusters, but routing can still miss a global neighbor.

Now we can inspect the work behind those times. The entries below are mean counts per query; a dash means the operation is not used.

Method	Opened rows	Rows scored	Split-mask words	Leaf-mask words	Prefix depths	Boundary searches
A	24,181	24,181	—	—	—	—
B	295,031	818	53,693	5,838	—	—
C	41,147	26,153	—	—	3.511	494.988

B opens a much larger set than A, but scores only about 0.28% of those rows across the query batch. It visits about 116 candidate sets per query, including splits and leaves. The extra bitmap work therefore accompanies a large reduction in full scoring, and the complete query is faster even than A’s separately chosen layout.

C scores about 63.6% of its own opened rows. It is faster than A here despite scoring slightly more rows than A’s selected configuration. That comparison also changes routing: C has 1024 clusters and 78 probes, versus A’s 4096 and 180. We cannot attribute the whole time difference to prefix pruning alone. Searching inside fixed clusters and comparing complete queries with different routing settings answer different parts of the question.

The stored timers measure routing and complete index search, but do not separately time scoring, each mask pass and queue maintenance. I therefore report those operations as separate work counts. One mask word, one full score and one boundary search are different operations. The complete-query times show the net difference, but do not give an independently measured time for each operation.

3.6 Where does the result change?

If we ask for 100 results, the conclusion changes. The table below uses the same 99% recall requirement for every method and reports the repeated median of its selected setting.

Collection/model	Bits	A (ms)	B (ms)	C (ms)
MS MARCO/Nomic	64	1.915	2.071	1.875
MS MARCO/Nomic	256	13.453	13.866	14.645
MS MARCO/Nomic	768	45.088	45.217	45.196
Quora/Qwen	32	0.400	0.439	0.404
Quora/Qwen	256	3.564	3.721	4.039
Quora/Qwen	1024	22.863	23.098	22.930

Every entry reaches at least 99% recall again in its repeats. At these selected settings, B does no splitting and C starts at depth zero. Within each row, all three methods use the same 4096-cluster layout, probe count and number of document scores. C has a slightly lower median on Nomic64, but it avoids no scores there. Small timing differences between these scan-like paths do not establish a pruning advantage.

The earlier Qwen32 top-100 lead for B disappears when A also receives the binary-trained layouts. With the stronger routing, A opens 380 clusters and scores about 50,865 rows at 0.400 ms; B and C score the same rows at 0.439 and 0.404 ms. For one result, however, B’s pruning advantage remains, and C also beats the best tested A setting, although B is faster overall. This is why I compare each method with A’s own selected settings rather than only holding A at one convenient probe count.

Code length. The 32-bit result answers a different ranking question from the full 1024-value embedding. To measure that difference, I find the best full-float score over the entire corpus, then rescore each width’s exact binary top 100 using those full vectors. “Agrees” means its score is within 10^{-5} of the full-float best; equal-score documents need not have the same ID.

Model/collection	Bits	Binary top 1 agrees	Best score reached within binary top 100
Qwen/Quora	32	14.0%	73.0%
Qwen/Quora	256	75.8%	100.0%
Qwen/Quora	1024	88.6%	100.0%
Nomic/MS MARCO	64	17.6%	65.6%
Nomic/MS MARCO	256	48.0%	97.2%
Nomic/MS MARCO	768	70.2%	99.9%

For Qwen32, even the exact binary top-1 answer reaches the full-float best score on only 14% of queries. Keeping the binary top 100 raises that to 73%, compared with 100% at 256 bits. So a faster, high-recall search over 32-bit codes does not by itself preserve full-embedding results. These are score-agreement diagnostics, not human relevance scores or measured end-to-end quality for B or C. An approximate search may also miss some of those exact binary candidates.

Exact code matches. A separate global experiment removes routing entirely and requires the exact best binary result. I select one setting per method on all 1,000 Qwen32 queries, then split those same repeated observations by whether the query’s full 32-bit sign pattern exists in the corpus. There are 82 such queries and 918 without a matching code. The settings are not retuned for either group: B uses leaf 2048 with no node limit, and C starts at 32 with target zero and the exact-stop bound.

Query group	Queries	A (μ s)	B (μ s)	C (μ s)
All queries	1000	2481.667	124.084	1163.521
Complete code present	82	2481.250	128.521	4.875
Complete code absent	918	2481.708	123.750	1170.730

All three methods return the exact binary result in every group. On the 82 matching-code queries, C scores only 1.68 rows on average and stops after its first pair of boundary searches. This is a real case where direct prefix lookup helps. A matching binary code does not imply identical text. But on the remaining 918 queries, it scores about 303,177 rows on average while widening the prefix. B therefore remains faster over the full 1,000-query batch. The 4.875 μ s result belongs to that 8.2% subgroup under the same global settings; it is not C’s overall time or a comparison with separately tuned subgroup indexes.

3.7 Checking the calculations

I checked stored field sizes and operation counts separately from the timings. Across 13 saved work-check reports, all 890 index-storage checks match the formulas, including padding at cluster boundaries; the saved query-count checks report no failures. These checks do not assign a constant time to a mask word or document score.

The score gap helps explain why short codes behave differently. For each Qwen query, I divide the gap between the ideal binary score Q and the true best score by Q . The median gap is 2.60% at 32 bits, 14.48% at 256 bits and 16.15% at 1024 bits. When a retained score is close to Q , a smaller mismatch penalty can rule out a branch. The short-code result makes such pruning possible earlier; the representation table shows the separate cost of changing the ranking problem.

At the other extreme, one completed global Nomic768 setting with $K = 100$, leaf 2048 and budget 8192 scores all one million rows. It also visits 21.57 million split-mask words and 21.46 million final-mask words per query on average: about 43.03 million word visits without avoiding a full score. More available RAM would not remove that work.

Stopping depth. The exact-stop option uses the ideal score $Q = \sum_i |q_i|$ and the worst of the K retained scores, τ . At positive prefix depth ℓ , every unseen opened row disagrees with at least one retained sign. Its score is therefore at most

$$u_\ell = Q - 2 \min_{1 \leq i \leq \ell} |q_i|.$$

C can stop when $\tau > u_\ell$, with the code’s rounding margin and tie rule. This proves the result within the opened clusters; it does not correct routing misses.

For the global Qwen32 exact-stop runs, let y_q be the deepest stored prefix whose unseen bound is below the saved exact best score, or zero if none qualifies. A walk starting at x then ends at $\ell_q = \min(x, y_q)$. Counting the corresponding sorted prefix range gives the number of scored rows; the visited-depth and lookup counts follow from $x - \ell_q + 1$. All four predictions match all 3,000 query records: the same 1,000 queries at starts 1, 4 and 32.

Start x	Mean final depth	Mean rows scored	Mean visited depths	Mean boundary searches
1	0.665	353,270.1	1.335	2.000
4	1.497	290,211.1	3.503	6.336
32	4.505	278,316.6	28.495	56.320

For $K = 1$, 665 queries prove a stop above the root. Starting at 32 reduces the mean number of scores, but requires many more boundary searches. The prediction uses reference scores only in this offline check; actual search uses the scores it has found. The saved $K = 100$ predictions were not measured in this global run, so they are not included in the 3,000 matching records.

Memory at the selected settings. For the Qwen32 top-1 settings, the verified native fields plus router storage and the largest repeated-process memory peak are:

Method	Native and router fields (MB)	Repeated-process peak (MB)
A	8.924	77.971
B	10.463	72.139
C	10.598	77.529

MB here means one million bytes. The field count includes codes, IDs, bitplanes or prefix keys, and router centroids and labels. It excludes unused capacity and other process memory, so it must not be confused with the measured peak. Even the selected Nomic768 B setting uses 225.1 MB of stored fields and a 505.6 MB process peak. The 505.6 MB peak is the largest across all selected repeated settings. Every one also fits a 64 GB or 128 GB allowance, so raising the feasibility limit alone leaves these recorded comparisons unchanged. This does not predict the performance of a larger, untested index.

Random visits. I also held the Qwen32 layout at 64 float-trained clusters, opened 16, and kept leaf size 32 while varying B’s exploration probability. The following top-1 results use three seeds per probability. Recall is their mean and time is the median of their query-time medians.

Probability	Budget 128		Budget 8192	
	Recall	ms	Recall	ms
0	87.30%	0.0460	95.30%	0.0766
0.1	83.33%	0.0479	95.30%	0.0857
0.2	70.90%	0.0490	95.30%	0.0959

With the smaller budget, random visits change which branches are processed before the limit, and recall falls. With the larger budget, recall stays at 95.3% while time increases. The Nomic64 runs show no consistent time gain at matching recall either. This experiment does not establish that randomness can never help; it does not improve the tested tradeoff here.

3.8 Code and reproduction

The C++ scorer is shared by all three methods. A and B are in `cpp/index.cpp`; C is in `cpp/prefix_index_v2.cpp`. The Python worker in `experiments/prefix_batch_worker_v2.py` times routing and search, records the work counts, and compares returned IDs with the exact references.

After preparing the arrays using `experiments/prepare_prefix_pools_v2.py`, we can run one complete setup with this Python flow:

```
import json
from pathlib import Path
from experiments import prefix_study_v2 as study
path = Path("configs/prefix-study-v2/qwen-quora-d256.json")
config = json.loads(path.read_text())
output = Path("results/my-quora-run")
study.prepare(config, output) # Build clusters and the schedule.
study.run(output)             # Measure the declared settings.
study.summarize(output)        # Select by recall and time.
study.repeat(output)           # Repeat those choices three times.
```

Full commands, input hashes, selected settings and raw query records are linked from `results/prefix-study-2026-09-13/README.md`. The data and encoding recipe is in `docs/expanded-pool-preparation.md`. The root `README.md` maps the paper to its code, including the exact-stop prediction and full-float agreement checks.

4 Limitations and next steps

The timings describe this CPU implementation. All three methods reuse the same C++ lookup-table scorer, which adds entries using float64. Exa’s production system and its SIMD kernels are not reproduced here. A faster scoring kernel can change the balance between scoring documents and maintaining an index search.

I hold the collection, model and code length fixed when choosing each method’s parameters. Shorter codes change the reference ranking, so their binary recall does not establish the same retrieval quality as a longer representation. The source datasets provide human relevance labels for that separate evaluation.

These are static indexes built before querying. The experiment does not measure document updates or concurrent requests. In particular, keeping rows sorted still costs work when new documents are added. The memory formulas count the binary search index and its query buffers; an application that keeps uncompressed vectors or document text in RAM must add that storage.

Prefix expansion also appears in LSH Forest [8] and PUFFINN [9]. Those methods use multiple hash indexes and different stopping rules. Their recall guarantees do not transfer to a single prefix

in the embedding’s original coordinate order.

5 Conclusion

For Qwen32 with one returned result, B takes 0.081 ms at 99.2% binary recall, versus A’s 0.195 ms and C’s 0.169 ms at 99.0%. All methods can choose binary-trained clusters. With 100 results at the 99% target, the selected B and C settings instead become scans. C has a narrower exact-search benefit: it takes 4.875 μ s on the 82 queries whose complete code exists, while B remains faster over all 1,000 queries.

These results identify useful tested settings, not a universal optimum. Qwen32’s exact binary top 1 agrees with the full-float best score on only 14% of queries, so the short-code speed does not preserve full-embedding quality. For a fixed representation and recall requirement, the decision remains whether the scores avoided pay for the added mask or prefix work. The measured counts let us check that explanation directly.

References

- [1] The Exa Team. *How we built a web-scale vector database*. 17 December 2024. <https://exa.ai/blog/building-web-scale-vector-db>.
- [2] Y. Malkov and D. Yashunin. *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. 2016; revised 2018. <https://arxiv.org/abs/1603.09320>.
- [3] A. Kusupati et al. *Matryoshka Representation Learning*. NeurIPS, 2022. <https://arxiv.org/abs/2205.13147>.
- [4] Nomic AI. *nomic-embed-text-v1.5*. Model card, accessed 13 September 2026. The saved manifest identifies the exact model revision. <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>.
- [5] Y. Zhang et al. *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*. 2025. <https://arxiv.org/abs/2506.05176>. Encoding recipe: <https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>.
- [6] P. Bajaj et al. *MS MARCO: A Human Generated MACHINE Reading COMprehension Dataset*. 2016; revised 2018. <https://arxiv.org/abs/1611.09268>. Passage-ranking data: <https://github.com/microsoft/MSMARCO-Passage-Ranking>.
- [7] N. Thakur et al. *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. NeurIPS Datasets and Benchmarks, 2021. <https://arxiv.org/abs/2104.08663>. Dataset catalog: <https://github.com/beir-cellar/beir>.
- [8] M. Bawa, T. Condie and P. Ganesan. *LSH Forest: Self-Tuning Indexes for Similarity Search*. WWW, 2005, pp. 651–660. <https://www.cs.princeton.edu/courses/archive/spring06/cos592/bib/LSHForest-bawa05.pdf>.
- [9] M. Aumüller, T. Christiani, R. Pagh and M. Vesterli. *PUFFINN: Parameterless and Universally Fast Finding of Nearest Neighbors*. ESA, 2019. <https://doi.org/10.4230/LIPIcs.ESA.2019.10>.